OAK RIDGE National Laboratory

Bash Scripting

Christian Goethert, Chance Loveday

Oak Ridge Leadership Computing Facility (OLCF)
Oak Ridge National Laboratory (ORNL)

July 8, 2026

Crash Course Material

- SSH into Odo / clone the repo if you haven't already
- cd ~/foundational_hpc_skills/intro-to-bash-scripting
- Following along interactively during presentation

- Walkthrough on github repo (use the README in browser)
  - Step-by-step tutorial
  - Premade scripts
  - Bonus challenges at the end'

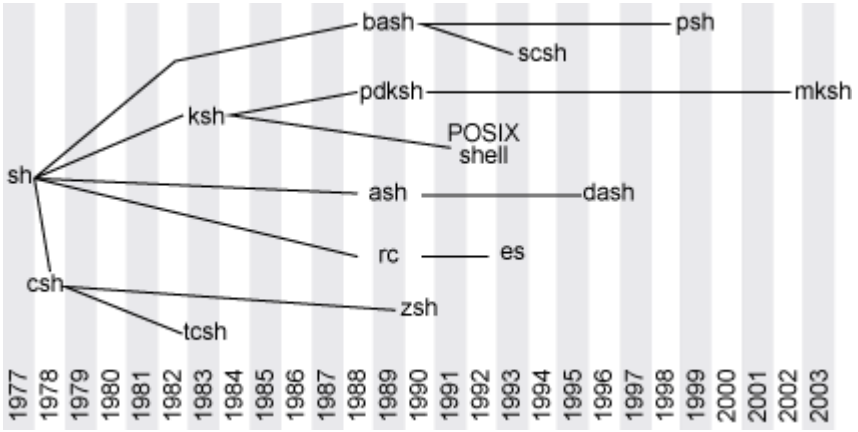
Run the following command:

git clone https://github.com/olcf/foundational_hpc_skills/

What is Bash?

Bourne-Again SHell — command interpreter, shell, and scripting language

- Created by Brian Fox around 1988; still under active development
- Used almost everywhere on Unix-based systems and macOS
- Its core job: managing and operating the OS directly
- Best learned as a supplement to a language you already know, not as a first language



What is Bash?

Great, but how does Bash compare to other languages?

- We've actually used Bash!
- Bash is the UNIX-based shell most of us have been using through Odo and during the HPC Crash Course

Handle With Care

•No "undo" when moving, copying, or deleting files.

•Not a general-purpose language — weak data structures, primitive error handling.

•Great for talking directly to your computer, fast.

A Quick Preview

Most of this lesson will show you how to take the logic you've seen in Python and apply it in a Bash environment

- Different Types of Variables
- Arithmetic
- Comparisons
- Strings
- Conditionals (if/elif/else)
- Loops
- Functions

Let's Review Linux Commands

Navigation

- `pwd`
- `cd`
- `ls` (-a, -l flags)
- `mv` (-r flag for folders)

Files

- `mkdir`
- `touch` - creates file
- `vi` for vim creating file
- `rm` (-r flag for folders)
- `cp` (-r flags)

More uses

- `cat` - concatenates
- `head` - first 5 lines of file
  (-n ___ for the first ___ lines)
- `tail` - last 10 lines of file

Search

- `grep` - search inside file
- `find` - Search for files within specified location

Back to Basics: Hello World!

- .sh extension = Shell file (not required to run)
- Alternatively, use .bash to be more specific

Run hello.sh

Great! Our program is saying “Hello” and “Goodbye”

- Looking at the code using Vim, two things should pop out.
  - Shebang: Specifies Bash as working shell
  - Prints to the terminal

```
#!/bin/bash


echo "Hello, World!"
echo "..."
echo "..."
echo "..."
echo "..."
echo "..."
echo "Goodbye, World!"
```

Back to Basics: Hello World!

This time run `hello.sh` and send the output to a text file called `output.txt`.

```
sh hello.sh > output.txt
```

Warning: This will erase any existing data in output.txt if the file exists and create a new file otherwise.

Instead, you can concatenate the text to the end of `output.txt`.

```
sh hello.sh >> output.txt
```

Run hello.sh

- '>' = creates/overrides output file
- '>>' = Adds to end of existing output file

Back to Basics: The | Operator

The pipe operator allows you to "pipe" output from one program as the input for another program

Word Counter Program

-w: Counts # of words
-m: Counts # of chars
-l: Counts # of lines

- You could pipe command1 | command2 to command3 all in one line!

```
command1 | command2 | command3
```

- We will explore more functions to pipe into in **String Manipulation**

```
[cloveday@login2.odo 01-hello]$ sh hello.sh | wc -w
9
```

Variables

In general, there are three types of variables:

Scalar

A single value, such as a string or number

Array

A collection of mixed values

- Functionally equivalent to lists and dictionaries in other languages

Environment

System-wide vars like $USER, $HOME, and $PATH

Declaring typed vars:

```
declare my_var -{flag}
```

Common `declare` Flags:

Flag	Description of flag operation
-i	Declare as an integer. Arithmetic expressions inherently handled and evaluated.
-a	Declare as an indexed array. This is assumed if not declared. More on this in **Array Variables**
-A	Declare as an associative array. More on this in **Array Variables**
-r	Declare as read-only, meaning its value cannot be changed.
-x	Mark a variable for export to the environment. More on this in **Environment Variables**
-p	Print the attributes and value of each variable. Useful for checking variable types

Variables: Scalar Variables

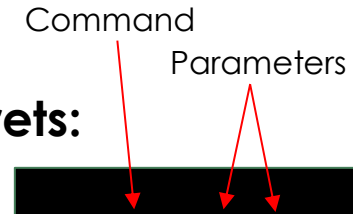
- What we think of as basic variables
  - Strings, ints, floats, … but they read in as strings
  - For this reason, Bash vars are generally considered 'untyped'

- Do not include spaces when assigning values

How Bash interprets:

```
my_num=3

# Stores my_num as 3
```




```
my_num = 3

-bash: my_num: command
not found
```

Variables: Scalar Variables

- Encapsulation - The method of enclosing vars in {} when printing
  - Encapsulation is not strictly necessary but is more precise

General Print Syntax: `echo "My var is ${my_var}"`

```
my_num=3
echo "My num is ${my_num}"
```

```
my_string="my string"
echo "My string is $my_string123"
```

≠

```
my_string="my string"
echo "My string is ${my_string}123"
```

Try this yourself in variables.sh

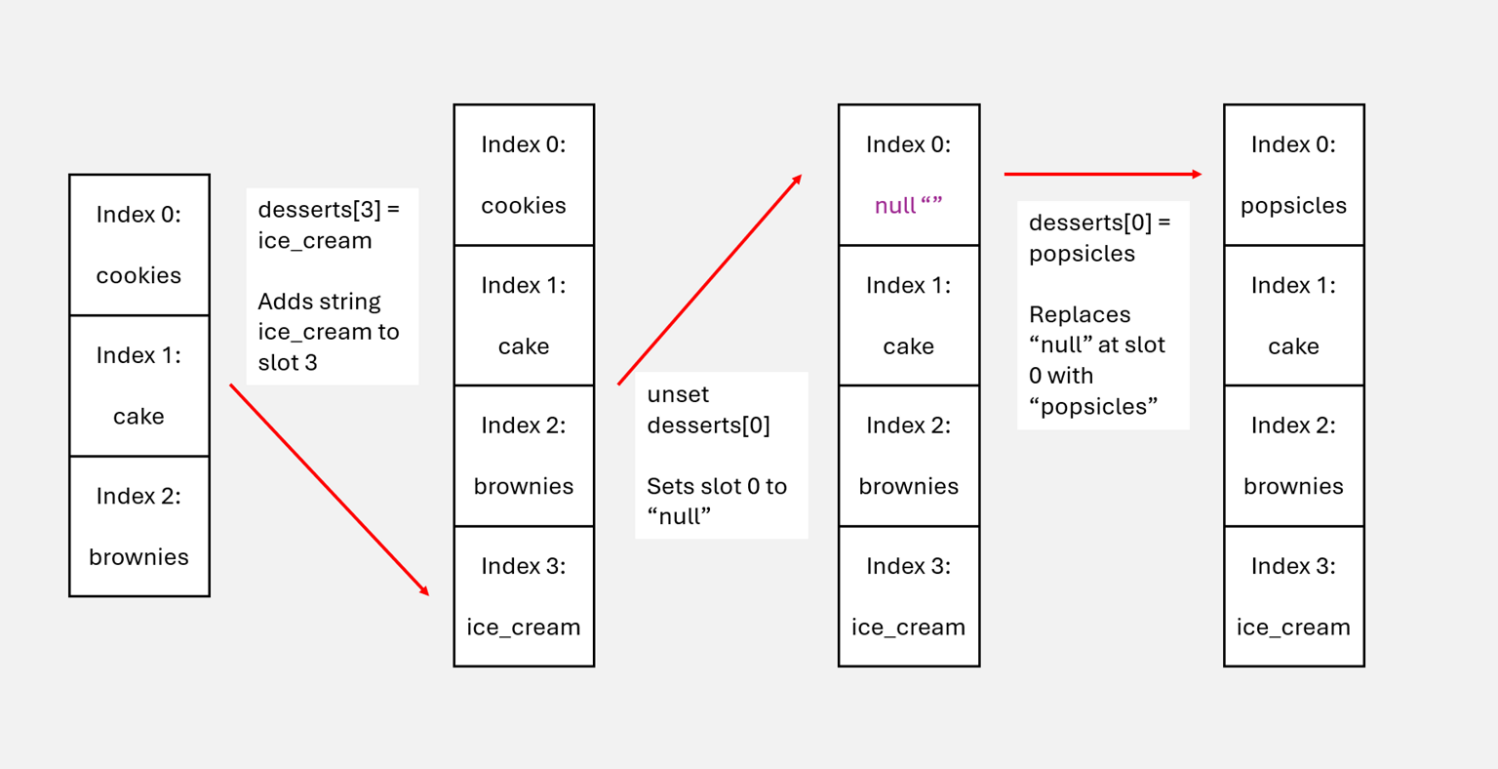
Variables: Arrays

- Arrays get their own datatype in Bash
  - Arrays typically store one datatype
  - However, Bash arrays can store mixed types, but you could argue that all vars are stored as strings anyway

```
my_array=(data1 data2 data3 data4)
```

- Let's look at array_variables.sh to explore syntax
  - Arrays are enclosed with `()`, not `[]`
  - Items are separated by a single space instead of commas
  - Arrays don't change size when you remove something
  - Use `unset my_array[n]` to clear the nth index

Variables: Arrays



Variables: Associative Arrays

- Associative arrays are Bash's equivalent of a dictionary
  - They are indexed by keys (strings)

Declare by using `declare my_array -A`

```
#!/bin/bash

declare -A colors
colors["red"]="Hex code: FF 00 00"
colors["blue"]="Hex code: 00 00 FF"
echo ${colors["red"]}  #Output:
"Hex code: FF 00 00"
```

Important Notes

- Associative arrays only work with Bash 4.0+
- Arrays are stored in memory, so they don't scale to large datasets

Variables: Environment Vars

Environment Variables

ENV Var	Variable description
$USER	Gives username of the current user (you, I hope)
$HOME	Gives filepath to your home directory
$PATH	Gives your command search path
$SHELL	Gives the location of your shell
$PWD	Gives the current working directory you are in
$LANG	Gives the default system language

- Bash is **a shell 1st** and **a programming language 2nd**
- Env. vars give you a lot of power over your operating system environment
- Be careful!

Variables: Special Vars

Special Variables

$0	Script name
$1..$N	Positional parameters
$#	Parameter count
$@ / $*	All params, separate / combined
$?	Exit status of last command
$$ / $!	PID of script / last background job

Variables: User Input

Say you don't want hardcoded data

- `Read <input var>` command = Takes in input from the user
- You can also direct input from an input file via <

```
read var1  #Basic form of reading input
read -p "Enter input: " var2
#You can send the -p flag to print a "prompt" to screen for input

read -p "Enter 3 items separated by a space: " l0 l1 l2
#You can have multiple inputs on one line
```

Arithmetic: Basic Syntax

- Every variable is stored as a string by default
- As a result, arithmetic requires specific syntax to interpret correctly

To add two numbers, a and b, together, surround the expression in $(()):

```
$((a + b))
```

```
a=14; b=2
echo "$((a + b))"   # 16
echo "$((a * b))"   # 28
echo "$((10 / 4))"  # 2 (!)

#Use bc operator for floating-point arithmetic
echo "scale=2; $a / $b" | bc
#3.33
echo "scale=3; $a / $b" | bc
  #3.333
```

bc Operator

- Notice that these operators cannot handle floats
- Via the | operator, bc handles float operations
  - scale = # of decimal points
  - `bc -1` to access trig and other mathematical functions

Arithmetic: More Operations

Fundamental Operations

Syntax	Effect
`$((a + b))`	Addition
`$((a - b))`	Subtraction
`$(a * b)`	Multiplication
`$((a / b))`	Division
`$((a ** b)`	Exponents
`$((a % b))`	Modulo Division

Assignment Operators

Syntax	Effect
`a=$((a += b))`	Incrementation by constant
`a=$((a -= b))`	Decrementation by constant
`a=$((a *= b))`	Multiply by constant
`a=$((a /= b))`	Divide by constant
`a=$((a %= b)`	Modulo by constant

Comparisons: From Python to Bash

Python

```python
# elif.py

# initialize a variable "x"
x = 1

# check to see if "x" is less than 1, 2, or 3
if x<1 :
    print('x is less than 1')
elif x<2 :
    print('x is less than 2')
elif x<3 :
    print('x is less than 3')
else:
    print('x is not less than 1, 2, or 3')
```

Bash

```bash
#!/bin/bash

read -p "Enter a number: " n

if [[ $n -lt 3 ]]; then
        echo "My number is less than 3"
elif [[ $n -eq 4 ]]; then
        echo "My number is 4"
elif [[ $n -eq 5 ]]; then
        echo "My number is 5"
elif [[ $n -ge 6 ]]; then
        echo "My number is 6 or greater"
else
        echo "My number is 3."
fi
```

Comparisons: How to Use Them

Bash has comparison operators like in Python and other languages

```
[[ 5 -lt 3 ]]
#Running this command on its own returns
nothing, but ...
echo "$?"
#Output: 1; 1 = False, 0 = True

[[ 15 -gt 7 ]] && echo "Success!!"
#Output: Success!!
```

Op	Meaning
`-eq / -ne`	Equal / not equal
`-lt / -le`	Less than / or equal
`-gt / -ge`	Greater than / or equal
`&& / || / !`	AND / OR / NOT

Some important syntax

- Use spaces for comparisons
- There are several use cases for enclosing these expressions, but double brackets is the most modern syntax → `[[ expression ]]`
- No booleans → represented as `0` and `1` instead
- The examples above are reserved for in-line commands but are still useful to know

Comparisons: If/Elif/Else

- Bash uses start/stop indicators rather than Python's use of indents
  - then = Start
  - fi = Stop

```
read -p "Enter a number: " n
if [[ $n -lt 3 ]]; then
  echo "less than 3"
elif [[ $n -eq 4 ]]; then
  echo "it is 4"
else
  echo "something else"
fi
```

- Run ex1.sh
- For another example, run ex2.sh

Strings: String Manipulation

Slicing

- Similar to slicing in Python

```
str1="Hello, World."
echo ${str1:0:5} #Output: “Hello”

echo ${str1: -6} #Output: “W”
```

Length

- Add a # before the var

```
str2="mSD90sfmjo"
echo ${#str2}
#Echos the length of the string
```

Replace

- / = Replace first instance
- // = Replaces all instances

```
str3="I dont like bash.
          I dont really like bash."
echo ${str3/dont/really}
echo ${str3//dont/really}
```

Substrings

- Returns the string after and including the specified string

```
str4="Substrings are difficult."
echo ${str4%%are*}
#Cuts off the part of the string past
(and including) substring "are"
```

Strings: Case & Pattern Matching

Case/Esac

- A pattern-matching alternative to if/elif
- Similar to `case` or `match-case` statements

- Start indicator = `case`
- Ends each branch with a double semicolon ;;
- End indicator = `esac`

Run grades.sh for an example

Pattern matching

General Format:

```
[[ <item1> -<compare operator> <item2> ]]
```

`=~`	`Match against a regex`
`!=`	`Not equal to`
`-n / -z`	`Non-empty / empty string`
`< / >`	`Lexicographic order`

Loops: Refresh

Do you remember the two types of loops?

Bash loops have slightly different syntax

- `do`/`done` = start/stop indicators for loops

For Loops → Iterative

While Loops → Condition-based

Loops: For Loops

- Remember how Bash was primarily written as a shell?
  - That means Bash is more suited for iterating through file contents or arrays
  - It can still handle range-based loops in a style similar to C

Loop Through Contents

```
for file in file*; do
        echo "${file} contents:"
        cat $file
        echo ""
  done
```

Loop Through Ranges

```
read -p "Enter a number: " num
for (( i=0; i<=num; i++)); do
        echo $i
done
```

Loops: While (Until) Loops

- While loops in Bash work more like `until` loops
  - While loops in Python and C continue until the condition breaks
  - In Bash, these loops repeat until the condition **is** met

```
# while loop
while [[ "$entry" != "exit" ]]; do
  items+=("$entry")
  read -p "Enter: " entry
done

# until: opposite of while,
# runs until condition is met
```

Functions: Refresh

- But there's a catch … a difference

```
myFunction () {
        echo "$1" #Gets 1st
parameter
        echo "$2" #Gets 2nd
parameter
        echo "$3" #Gets 3rd
parameter
        return 0
#Return value: typically used for
checking if the function executed
successfully (0) or not (nonzero)
}
```

- Parameters aren't declared in the function header
  - They're read via $1, $2, etc. inside the body
  - Values are sent in after function call, separated by a space
- Return signals an exit code (success/failure)
- Variables set inside a function are global by default → use local to contain them
- Functions can be nested and can call one another once defined.

More Useful Techniques

Defensive Bash

- Safeguards against user error

1) Enable "Strict Mode:"

```
set -euo pipefail
```

1) Check paths actually exist before using them in scripts
2) Surround vars in quotes during calls and assign them default values
3) Extra print statements for debugging

Parallel Task Execution

Say you want to run 2 [,3, 4, … as many as want] programs simultaneously.

- You can run programs in parallel!
  Ex: `sh welcome.sh &`
  `sh homework.sh &`
- This is a tease of how to run tasks in parallel on computing clusters

When to Use Bash Scripts?

When is it appropriate to use Bash vs. Python and other languages or frameworks?

➔ **Primary Goal:**
Automating + Streamlining Tasks

➔ Suitable for short scripts (<100 lines)
- Little error handling required
- File-Focused → files & directories
- No dependencies
- CLI interactions
- Custom configurations

➔ **Primary Goal:**
Data Analysis, Visualization, …

➔ Better for longer scripts
- Error handling
- Data-Focused → mathematical operations, plotting, …
- Advanced data structures and parsing (e.g., JSONs)
- API interactions
- Cross-platform needs

How Does Bash Fit Into HPC?

In High-Performance Computing

- Most HPC interfaces run on bash or bash-adjacent shells.
- Pairs naturally with job managers like Slurm for submitting and automating jobs.
- Can loop through job parameters, log results, and compare outputs in the background.
- Direct, low-level access to the file system and environment makes it ideal for setup and upkeep.

Practice Challenges

A set of hands-on challenges is in `08-practice-challenges`

Sample solutions are included

Homework

- A list of challenges are available in the `08-practice-challenges` directory (https://github.com/olcf/foundational_hpc_skills/tree/master/intro-to-bash-scripting)
- To access the challenges on the command line, you must change directories to the challenges directory:

```
$ cd ~/foundational_hpc_skills/intro-to-bash/08-practice-challenges
```

- If you are having trouble completing the challenges, potential solutions are provided in the solutions sub-directory

Additional Resources

- Bash Documentation: The Linux Documentation Project
- Advanced Bash Scripting Guide: https://tldp.org/LDP/abs/html/
- Practice Lessons: W3schools intro to bash scripting
- More Practice Lessons: GeeksforGeeks intro to bash
- Unix Shell Carpentry: Software carpentry